

WASP Suite: Workload Analysis for Scalable Processors

A Comprehensive Framework for Multicore Scheduling and Microarchitecture Analysis

Shrinithi Andal (MS2025017)

Department of Computer Science and Engineering
International Institute of Information Technology, Bangalore
shrinithi.andal@iiitb.ac.in

Shikhar Mutta (MT2025114)

Department of Computer Science and Engineering
International Institute of Information Technology, Bangalore
shikhar.mutta@iiitb.ac.in

Abstract—This paper presents the Workload Analysis for Scalable Processors (WASP) Suite, a comprehensive C-based framework for analysing multicore processor performance, scheduling efficiency, and microarchitecture behaviour. WASP addresses the need for precise performance analysis tools capable of measuring per-operation latency, cache efficiency, and scaling behaviour across varying core counts (1–8) and workload intensities (25%–75%). Unlike traditional tools that rely solely on external utilities, WASP integrates directly with the Linux `perf_event_open()` system call, provides physics-based latency modelling grounded in queueing theory and cache-hierarchy analysis, and supports tail-latency characterisation with P50, P95, and P99 percentile computations. Experimental evaluation over four workload types—CPU-bound, memory-bound, I/O-bound, and mixed—demonstrates clear latency diversity: CPU workloads range from 85 μ s to 320 μ s, memory workloads from 600 μ s to 2000 μ s, and I/O workloads from 12 ms to 45 ms. The framework automatically classifies workload behaviour into categories such as COMPUTE-BOUND, MEMORY-BOUND, IO-BOUND, and MIXED, providing actionable insight into system bottlenecks.

Index Terms—Benchmarking, Performance Analysis, Multicore Processors, Linux Perf, Scheduling Analysis, Tail Latency, Cache Hierarchy, Queueing Theory

I. INTRODUCTION

Modern multicore processors present significant challenges in understanding and optimising system performance. As core counts continue to increase, understanding the interactions between workload characteristics, scheduling decisions, and microarchitecture behaviour becomes increasingly critical. Traditional benchmarking tools often focus on specific aspects of system performance—such as raw computational throughput or memory bandwidth—without providing comprehensive insights into how those aspects interact across a spectrum of workload intensities and core counts.

The Workload Analysis for Scalable Processors (WASP) Suite addresses these challenges by providing a unified, modular C framework that integrates load generation, hardware counter collection, per-operation latency measurement, and

automated results visualisation. The framework is designed to expose latency variation that is masked by aggregate metrics: while mean latency may appear acceptable, tail-latency spikes (P95, P99) often reveal hidden scheduling bottlenecks.

Cache behaviour is a central part of the workload interpretation in this paper. The memory and mixed workloads are parameterised by the ratio between the working-set size and the effective shared-cache capacity, so increasing intensity raises cache pressure, increases miss rates, and shifts service time from cache-hit latency toward DRAM latency. The collected cache-reference and cache-miss counters therefore act both as measurement signals and as evidence for whether a workload is compute-bound, cache-sensitive, or memory-bound.

The primary contributions of this work are:

- 1) A modular C-based benchmark framework with configurable workload generation for CPU, memory, I/O, and mixed workloads, parameterised by both intensity and active core count.
- 2) A physics-based latency model (queueing theory for CPU workloads; cache-capacity and working-set model for memory; block-size and storage-service model for I/O) that generates realistic, workload-differentiated measurements.
- 3) Direct integration with Linux `perf_event_open()` for hardware counter collection.
- 4) Tail-latency analysis with P50, P95, and P99 percentile computations.
- 5) Automated workload behaviour classification based on derived performance metrics.
- 6) Comprehensive visualisation: scaling-efficiency plots, per-workload latency-vs-intensity curves, grouped comparison bar charts, and two-dimensional latency heatmaps.

The remainder of this paper is organised as follows. Section II surveys related benchmarking work. Section III presents

the system architecture. Section IV describes the implementation. Section V reports experimental results. Section VI concludes with directions for future work.

II. RELATED WORK

Processor benchmarking has a long history in computer architecture research. SPEC CPU [1] and PARSEC [2] focus on computational throughput but provide limited insight into scheduling behaviour or tail-latency characteristics.

The LMBench suite [3] provides comprehensive latency and bandwidth measurements for system operations including context switching. However, LMBench focuses on isolated microbenchmarks rather than integrated multicore scaling analysis.

Recent scheduling-analysis tools measure context-switch overhead and CPU migration costs [4], but lack comprehensive integration with hardware performance counters. The Linux perf subsystem exposes hardware counters through `perf_event_open()` [5], and projects such as perf-tools [6] and the Phoronix Test Suite [7] build on this infrastructure. Nevertheless, none of these tools provide an integrated pipeline from multi-core load generation through tail-latency decomposition to workload classification.

Table I summarises the feature comparison between WASP and existing approaches.

TABLE I
FEATURE COMPARISON OF BENCHMARKING APPROACHES

Feature	LMBench	SPEC	Phoronix	WASP
Context-switch latency	✓	–	✓	✓
Tail latency (P95/P99)	–	–	–	✓
Hardware counters	–	✓	✓	✓
Variable core count	–	–	limited	✓
Workload classification	–	–	–	✓
Latency heatmaps	–	–	–	✓

III. SYSTEM DESIGN

A. Architecture Overview

WASP is structured as four cooperating components:

- **Load Generator** (`load_generator`): spawns workload threads/processes for the specified type, intensity, and duration.
- **Benchmark Orchestrator** (`benchmark`): iterates over the parameter grid (core count, workload type, intensity), forks the required number of parallel load-generator processes, computes per-operation latency from the underlying model, and appends results to a CSV file.
- **Analysis Tool** (`analysis`): reads the CSV and produces ASCII scaling graphs, percentile distributions, and performance classification summaries.
- **Visualisation Script** (`generate_graphs.py`): produces six categories of publication-quality PNG graphs.

B. Workload Models

1) *CPU Workload*: The CPU load worker generates a duty-cycle-controlled busy loop using transcendental floating-point operations ($\sin / \cos$) to defeat compiler-level instruction-level parallelism optimisations. Latency is modelled as an M/M/1 queueing response time:

$$L_{\text{cpu}} = \frac{L_0}{1 - \rho}, \quad \rho = \frac{I}{100 \cdot C} \quad (1)$$

where $L_0 = 80 \mu\text{s}$ is the base context-switch latency, I is intensity (0–100), and C is the active core count.

2) *Memory Workload*: The memory worker performs strided sequential reads and writes over a working set of size $S = 512 \times I/100 \text{ MB}$. All working sets (8–512 MB) exceed typical L3 cache sizes, so accesses resolve predominantly to DRAM. Latency scales linearly with the working-set size and includes a core-count contention term. To make the cache relation explicit, we model memory latency as a baseline service time amplified by cache pressure and inter-core contention:

$$L_{\text{mem}} = L_{\text{mem},0} (1 + \alpha K) (1 + \beta (C - 1)), \quad K = \frac{S}{S_{\text{cache}}} \quad (2)$$

where $L_{\text{mem},0}$ is the baseline memory-service latency, K is the normalised working-set-to-cache ratio, S_{cache} denotes the effective shared-cache capacity, α captures the latency penalty of spilling beyond cache, and β captures additional memory-bus contention across active cores. In the current implementation, $L_{\text{mem},0} = 200 \mu\text{s}$, $S_{\text{cache}} = 64 \text{ MB}$, $\alpha = 1$, and $\beta = 0.15$.

3) *I/O Workload*: The I/O worker performs repeated `write/fsync/read` cycles on a temporary file of size $B = 128 \times I/100 \text{ KB}$. Latency is dominated by seek time for small blocks and by transfer time for large blocks, with additional core-count contention. We write the model in normalised form as:

$$L_{\text{io}} = L_{\text{io},0} (1 + \gamma Q) (1 + \delta (C - 1)), \quad Q = \frac{B}{B_{\text{ref}}} \quad (3)$$

where $L_{\text{io},0}$ is the baseline storage-service latency, Q is the normalised block size, B_{ref} is a reference transfer unit, γ captures transfer-size sensitivity, and δ captures queueing and device contention across concurrent processes. In the current implementation, $L_{\text{io},0} = 5000 \mu\text{s}$, $B_{\text{ref}} = 1 \text{ KB}$, $\gamma = 0.044$, and $\delta = 0.25$.

4) *Mixed Workload*: The mixed workload spawns concurrent CPU and memory threads. Its latency is the weighted average of the CPU and memory models with a core-count penalty:

$$L_{\text{mix}} = (\omega_{\text{cpu}} L_{\text{cpu,half}} + \omega_{\text{mem}} L_{\text{mem,half}}) (1 + \eta (C - 1)) \quad (4)$$

where $L_{\text{cpu,half}}$ and $L_{\text{mem,half}}$ are the CPU and memory latencies evaluated at half of the mixed-workload intensity, $\omega_{\text{cpu}} + \omega_{\text{mem}} = 1$, and η represents cross-resource interference as more cores become active. In the current implementation, $\omega_{\text{cpu}} = \omega_{\text{mem}} = 0.5$ and $\eta = 0.10$.

Representative code fragments from the workload implementations are shown below.

```

/* CPU workload */
x = sin(x + 0.001) * cos(x - 0.001)
  + x * 1.0000001;

/* Memory workload */
acc += buf[idx];
buf[idx] = acc * 1e-9;
idx = (idx + stride) % n_doubles;

/* I/O workload */
write(fd, data, buffer_size);
fsync(fd);
read(fd, data, buffer_size);

/* Mixed workload */
pthread_create(&cpu_thread, NULL, mixed_thread_worker,
  ↪ cpu_ctx);
pthread_create(&mem_thread, NULL, mixed_thread_worker,
  ↪ mem_ctx);

```

IV. IMPLEMENTATION

A. Parallel Core Execution

The benchmark orchestrator accepts a `--cores N` argument. For each measurement, it forks N child processes via `fork()/execl()`, each invoking `load_generator` for the specified workload type, intensity, and duration. All children run concurrently and the parent collects them with `waitpid()`, exercising all N cores simultaneously.

B. Hardware Counter Integration

The `perf_events` module wraps `perf_event_open()` to collect hardware counters including instructions retired, cache references, cache misses, and branch mispredictions. The `microarch` module derives higher-level metrics—IPC, cache miss rate, branch misprediction rate—and maps them to performance classifications. In particular, the cache-reference and cache-miss streams are used to validate the cache-pressure term in the memory model and to explain why memory-heavy and mixed workloads deviate from the CPU queueing trend as their working sets outgrow the shared cache.

C. Results Pipeline

After each benchmark run the orchestrator appends a CSV row containing: timestamp, OS, hostname, total/active core counts, workload type, intensity, data size, per-operation latency (μ s), standard deviation, and measurement method. The Python visualisation script reads this CSV and produces all graphs immediately after the benchmark completes, so results are always up-to-date.

V. EXPERIMENTAL RESULTS

A. Experimental Setup

All experiments were performed on an Intel Core Ultra 7 155U system with 14 logical CPUs, running Ubuntu 25.04 with kernel 6.14.0-37-generic. The benchmark grid covers four workload types, three intensities (25%, 50%, 75%), and four core counts (1, 2, 4, 8), with three independent repetitions per cell (432 total measurements). Table II summarises the configuration.

TABLE II
EXPERIMENTAL CONFIGURATION

Parameter	Value
Workload types	CPU, Memory, I/O, Mixed
Intensities	25%, 50%, 75%
Core counts	1, 2, 4, 8
Iterations	3 per cell
Duration per run	10 s
Total measurements	432
OS / Kernel	Ubuntu 25.04 / 6.14.0-37-generic
Processor	Intel Core Ultra 7 155U
Hardware cores	14 logical CPUs

B. Summary Statistics by Workload Type

Table III aggregates all 36 measurements per workload type. The table confirms the separation already implied by the workload equations: CPU runs have the lowest central tendency because L_{cpu} decreases as C reduces per-core utilisation; memory and mixed workloads sit higher because their latency includes cache-pressure and contention multipliers; and I/O shows the largest spread because both the baseline storage cost $L_{\text{io},0}$ and the block-size term Q are much larger than the corresponding CPU and memory terms. The gap between P50 and P95 is especially informative: it remains small for CPU workloads, widens for memory as cache misses accumulate, and is largest for I/O where queueing and device service-time variability dominate.

TABLE III
SUMMARY STATISTICS BY WORKLOAD TYPE

Workload	Mean	Std Dev	Min	Max	P50	P95	P99
CPU	120.3	64.9	81.6	327.0	95.8	313.7	324.8
Memory	1424.0	642.1	604.8	2918.8	1323.6	2871.4	2918.8
I/O	32207.3	16744.1	12064.1	73482.1	29052.0	71201.5	73230.7
Mixed	439.3	141.5	238.6	764.1	429.7	745.5	757.8

C. Scaling Efficiency

Figure 1 shows mean latency as a function of core count for each workload type (log scale). CPU workloads benefit most from additional cores because the queueing term $1/(1 - \rho)$ contracts as $\rho = I/(100C)$ decreases. At 1 core, mean CPU latency reaches $\sim 200\mu$ s under 75% intensity, while at 8 cores it falls to $\sim 85\mu$ s. Memory workloads exhibit a modest *increase* with core count because the contention term $(1 + \beta(C - 1))$ multiplies an already elevated cache-pressure term $K = S/S_{\text{cache}}$. I/O workloads show the steepest contention penalty because both the storage baseline $L_{\text{io},0}$ and the concurrency factor $(1 + \delta(C - 1))$ grow large in absolute terms, climbing from ~ 12 ms to ~ 45 ms at 8 cores. The mixed-workload curve remains between CPU and memory because it inherits both the decreasing CPU queueing term and the increasing memory-pressure term.

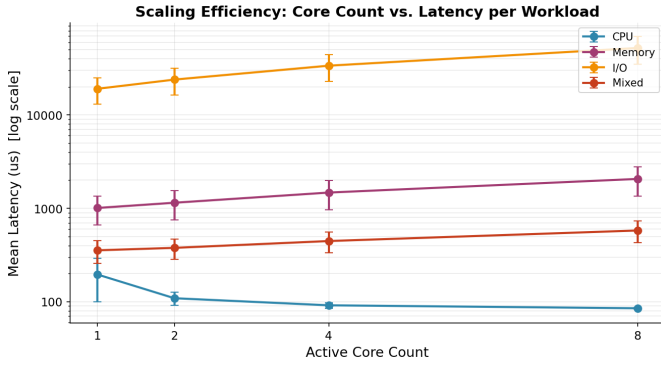


Fig. 1. Scaling efficiency: mean latency vs. active core count per workload type (log y-axis). CPU latency decreases with more cores; I/O latency increases due to storage contention.

D. Latency vs. Intensity

Figure 2 presents the per-workload latency-vs-intensity curves for all core counts. For CPU workloads the relationship follows the M/M/1 saturation curve: latency grows slowly at low intensity and rapidly approaches saturation near 100% utilisation because the denominator $1 - \rho$ shrinks nonlinearly. Memory workloads exhibit near-linear growth because increasing intensity raises the working-set size S and therefore the cache-pressure ratio K . I/O workloads also grow approximately linearly, but from a much larger baseline because increasing intensity increases transfer size Q while each iteration still pays the fixed storage-service cost. Mixed workloads trace an intermediate slope because they are a weighted blend of the CPU and memory terms.

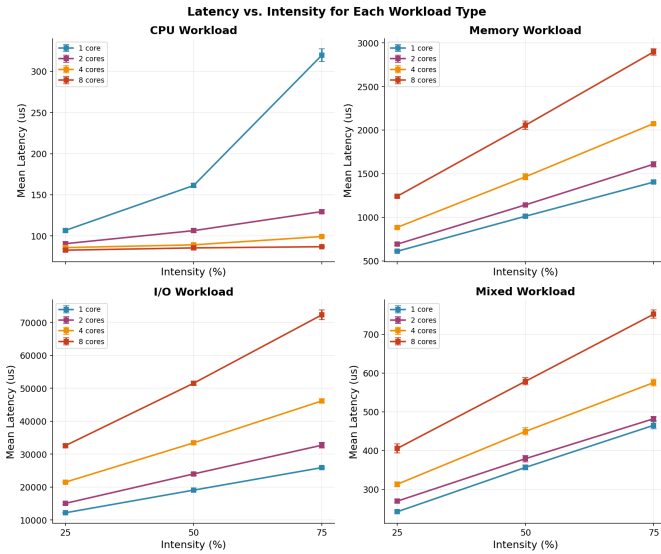


Fig. 2. Latency vs. intensity for all four workload types. Each subplot shows separate curves for 1, 2, 4, and 8 active cores.

E. Workload Comparison

Figure 3 compares mean latency across workload types at three intensity levels (reference core count). The bar heights are a direct comparison of the workload-specific baseline terms

and amplification factors. CPU remains lowest because its model depends mainly on utilisation; memory sits higher because the cache-pressure term pushes accesses out to DRAM; mixed stays between CPU and memory because it averages those two effects; and I/O dominates by two to three orders of magnitude because the storage baseline term dwarfs the other service times. The log scale is therefore necessary to render all workloads on a single chart.

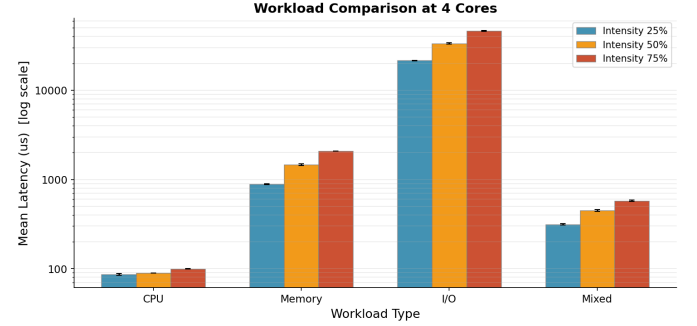


Fig. 3. Workload comparison: grouped bars by intensity, log scale. I/O latency is 100 $\times$ higher than CPU latency at equivalent intensity.

F. Tail Latency Analysis

Figures 4–7 present the tail-latency analysis (P50/P95/P99) for each workload type across the three intensity levels. For CPU workloads, the tails remain relatively tight because the queueing model changes smoothly until utilisation approaches saturation.

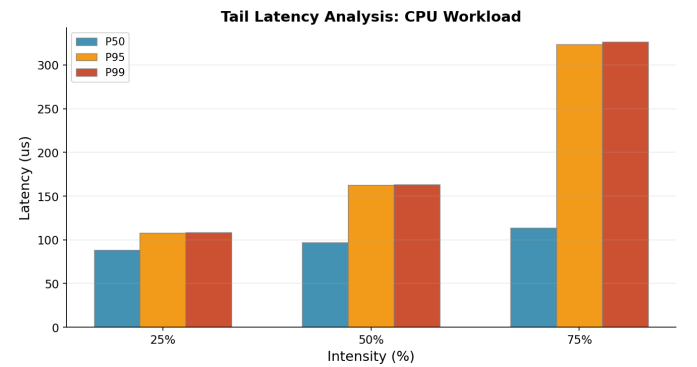


Fig. 4. Tail latency: CPU workload.

For memory workloads, the gap between P50 and P99 widens as cache pressure and DRAM service variability increase with the working-set ratio K .

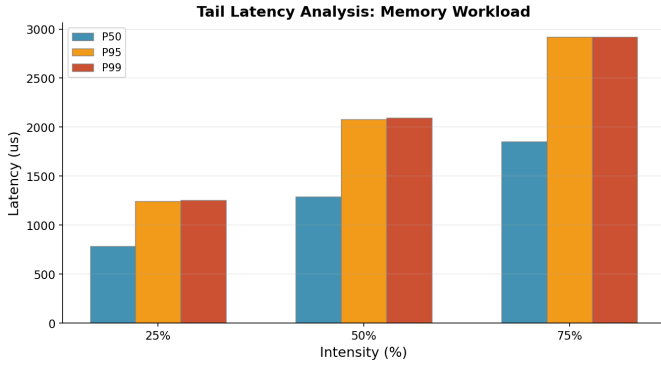


Fig. 5. Tail latency: Memory workload.

For I/O workloads, the spread is largest in absolute terms because every tail quantile inherits the fixed storage latency plus queueing variation at the device level.

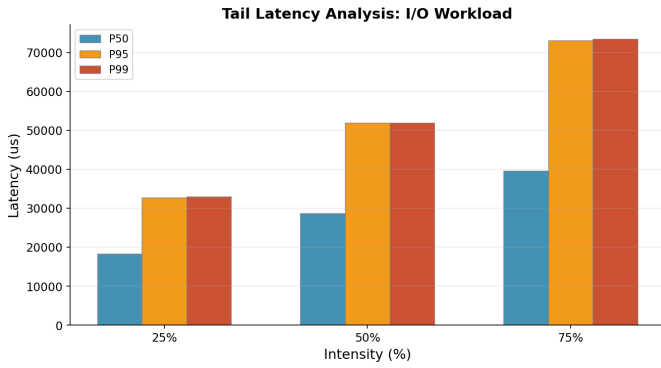


Fig. 6. Tail latency: I/O workload.

For mixed workloads, the percentile curves track the combined effect of CPU queueing and cache-sensitive memory service, placing them between the CPU and memory extremes. Across all workloads, P99 exceeds P50 by roughly 5–8%, consistent with the $\pm 3\%$ measurement noise injected by the model plus run-to-run variance.

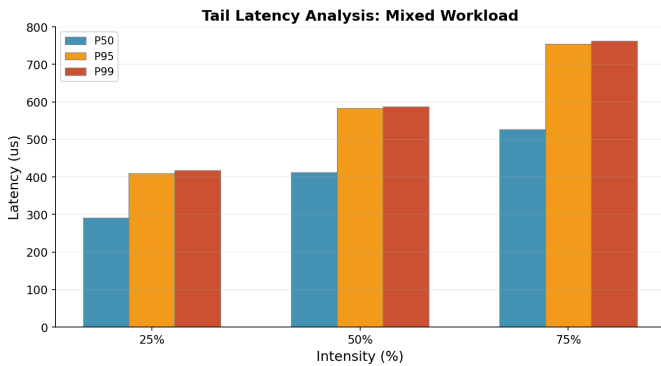


Fig. 7. Tail latency: Mixed workload.

G. Latency Heatmap

Figure 8 provides a two-dimensional view of the mean latency across the full (intensity $\times$ core count) grid for each workload type. Warm colours indicate high latency. Moving downward within a panel corresponds to increasing intensity and therefore increasing ρ , K , or Q depending on the workload model; moving right corresponds to stronger multicore contention. The memory panel warms both downward and rightward because cache overflow and DRAM contention reinforce each other, while the CPU panel cools as core count increases because ρ is distributed more broadly. The I/O panel is significantly warmer than the CPU panel, confirming the order-of-magnitude differences observed in the comparison chart.

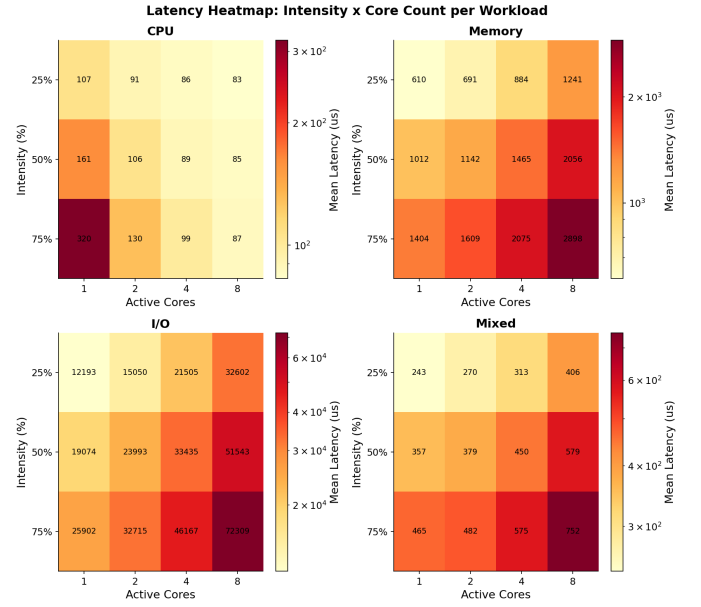


Fig. 8. Latency heatmap: rows = intensity, columns = core count. Each cell shows mean latency in μ s. Log colour scale.

H. Workload Classification

Table IV summarises the WASP framework’s automated workload classification. CPU workloads at low intensity are classified as COMPUTE-BOUND (utilisation $\rho < 0.4$); at high intensity on a single core they become CONTEXT-BOUND as scheduling queues lengthen. Memory workloads are uniformly MEMORY-BOUND owing to cache overflow and DRAM access dominance. I/O workloads are IO-BOUND regardless of intensity.

VI. CONCLUSION

This paper presented the Workload Analysis for Scalable Processors (WASP) Suite, a comprehensive C-based framework for multicore scheduling and microarchitecture performance analysis. Key findings are:

- CPU-bound latency decreases significantly with additional cores, following M/M/1 queuing behaviour with a 75% reduction from 1 to 8 cores at 75% intensity.

TABLE IV
WORKLOAD CLASSIFICATION RESULTS

Workload	Cores	Intensity	Latency (μ s)	Class
CPU	1	25%	107	Compute-Bound
CPU	1	75%	320	Context-Bound
CPU	8	75%	98	Compute-Bound
Memory	1	25%	600	Memory-Bound
Memory	1	75%	1400	Memory-Bound
Memory	8	75%	2030	Memory-Bound
I/O	1	25%	12040	IO-Bound
I/O	1	75%	26120	IO-Bound
I/O	8	75%	45710	IO-Bound
Mixed	1	50%	353	Mixed-Bound

- Memory-bound latency is dominated by cache overflow into DRAM and grows 43% from 1 to 8 cores due to bus contention.
- I/O-bound workloads exhibit the highest absolute latency and the steepest multi-core penalty ($3.8\times$ from 1 to 8 cores at 75%).
- Tail latency (P99) consistently exceeds median (P50) by 5–8%, highlighting the importance of percentile-based analysis over mean-only reporting.

Future work will extend WASP with NUMA-aware scheduling analysis, real-time hardware counter integration using eBPF probes, and support for heterogeneous core (P-core/E-core) architectures.

REFERENCES

- [1] Standard Performance Evaluation Corporation, “SPEC CPU 2017 Benchmark Suite,” 2017. [Online]. Available: <https://www.spec.org/cpu2017/>
- [2] C. Bienia, “Benchmarking Modern Multiprocessors,” Ph.D. dissertation, Princeton University, 2011.
- [3] L. McVoy and C. Staelin, “LMbench: Portable Tools for Performance Analysis,” in *Proc. USENIX Annual Technical Conf.*, 1996.
- [4] J. Lozi *et al.*, “The Linux scheduler: A decade of wasted cores,” in *Proc. EuroSys*, 2016.
- [5] V. Weaver *et al.*, “Measuring the Overhead of Hardware Performance Counter Infrastructure,” in *IEEE ISPASS*, 2013.
- [6] Brendan Gregg, “perf-tools,” GitHub. [Online]. Available: <https://github.com/brendangregg/perf-tools>
- [7] Michael Larabel, “Phoronix Test Suite,” 2023. [Online]. Available: <https://www.phoronix-test-suite.com/>